



**FACULTY
OF ELECTRICAL
ENGINEERING
CTU IN PRAGUE**

Drone Detection & Localization

Student kickoff: the project, the theses and how we work

Martin Maxa Kamil Herman Jakub Svatoš

Czech Technical University in Prague
Faculty of Electrical Engineering, Department of Measurement

Kickoff meeting, autumn 2026

1. **What we build:** drone detection as a system
2. **The five theses:** who does what
3. **How we develop:** repo, docs, devcontainers, git, AI
4. **How we plan:** Kanban on GitHub Projects (live)
5. **Next steps**

Joint meetings

Only three per semester:

🚩 kickoff (today)

🕒 mid-semester

🏁 end of semester

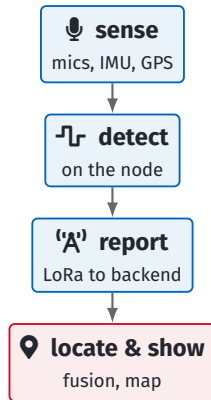
Everything else happens in individual consultations, whenever you need one.

Why drone detection?

- Small drones are cheap, everywhere, and hard to see.
- Many sensing principles exist: radar, RF, optical, thermal, **acoustic**.
- Acoustic units are **cheap, passive and low-power**, so we can build a network of them.

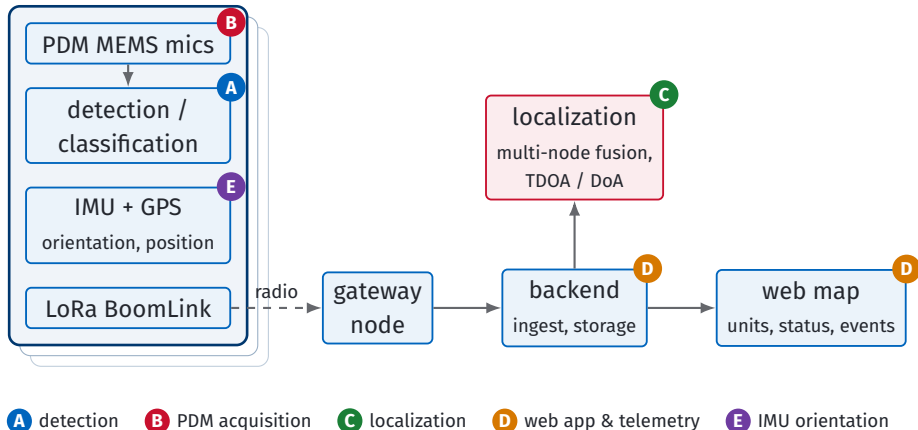
Our goal

A **working end-to-end system**: sensor units in the field, a network, a backend and a map. We can then push detection as far as it goes.



The system and where your thesis fits

Sensor nodes (STM32H563)



What we already know is hard

- **Range:** sound attenuates fast and drones often fly high.
- **Noise:** wind, traffic, and the drone's own ego-noise.
- **Low power:** the node runs on a battery, so every mW counts.

Where it can go next

- **Microphone arrays:** direction of arrival, beamforming, more gain.
- **Sensor fusion:** acoustic + **RF detection**.
- Distributed networks of many cheap nodes.

🦉 We **will** hit dead ends. That is fine as long as we measure them and write them down.

Right now the priority is the **system itself**: make it work end to end first.

Ⓐ Acoustic Drone Detection on the STM32 Platform

 Jan Amos Vorel  supervisor Martin Maxa

Goal

Survey and compare features (spectral, MFCC / log-mel) and classifiers (classic ML, tiny NNs) for deciding *drone* / *no drone* from short audio segments, with embedded deployment in mind.

Where it plugs in

The **detection** block on the sensor node. It builds on the existing MFCC + classifier code in `fw/common/boomdetect` and `fw/bom-stm32node/App/detect`.

Key deliverables

- PC pipeline on real recordings
- accuracy, false alarms, behaviour under noise
- memory, compute and latency budget
- optional: feature extraction on STM32
- split: what runs on the node vs. centrally

 Ivan Dyryvolkov  supervisor Martin Maxa

Goal

Record 4–6 PDM MEMS microphones at once on the STM32H5 using SAI in PDM mode + DMA, and show that the channels are **time-synchronous**.

Where it plugs in

The **microphone front-end** of the node and the enabler for microphone arrays. It starts from the single-mic SAI + GPDMA path in fw/bom-stm32node (mic.c, pdm_pcm.c).

Key deliverables

- clocks, SAI pins, DMA configuration
- buffer layout and de-interleaving to channels
- measured inter-channel delay
- data rate, CPU load, memory
- recommendations for DoA algorithms

 Vojtěch Beneš  supervisor Martin Maxa

Goal

A systematic survey of how drones are localized (radar, RF, optical, thermal, **acoustic**) with focus on direction of arrival, range estimation and distributed sensors.

Where it plugs in

The **localization** block on the central side, and input for where we take the project next. Related code: `scripts/tdoa_estimation`.

Key deliverables

- structured comparison of methods
- acoustic DoA, TDOA, multilateration
- distributed-sensor approaches
- promising directions for us
- simulation / public data / experiment

 **Martin Voplakal**  supervisor Jakub Svatoš

Goal

A web application for the network of detection units: the server receives, validates and stores telemetry, and a client map shows it in real time.

Where it plugs in

The **backend + web map** on the central side. A first version lives in apps/api-backend and apps/device-web, but the technology choice is yours.

Key deliverables

- survey of protocols and databases for telemetry
- backend architecture (battery, GPS, signal, events)
- choice of map API (Leaflet, MapLibre, Mapy.cz, ...)
- responsive real-time UI

 **Lumír Blažek**  supervisor Kamil Herman

Goal

Know the node's **pitch, roll and yaw** so that acoustic measurements can be converted to global coordinates. This means accelerometer + gyro fusion with GPS drift correction.

Where it plugs in

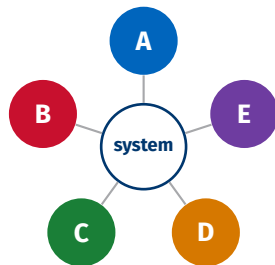
The **IMU + GPS** block of the sensor node. Without orientation, a direction of arrival from the node cannot be placed on the map.

Key deliverables

- I2C/SPI + DMA driver (e.g. LSM6DS3)
- calibration and fusion algorithm
- tamper / fall detection
- SD-card logging, field tests
- low-power modes

Independent, but connected

- The topics are designed so that **nobody waits for anybody**.
- Each thesis can be finished and defended on its own.
- But the outputs meet in one system, so **know what the others do**.
- Share what you learn: a working driver, a dataset or a dead end helps everyone.



One repository for everything

fw/	firmware (STM32 node, ESP32, shared libs)
apps/	backend, web, Discord bot
hw/	hardware (Altium)
scripts/	DSP experiments (TDOA, filters, ...)
projects/	self-contained student projects
papers/	publications and talks
docs/	documentation (MkDocs)
.devcontainer/	dev environments

Why a monorepo?

- firmware, backend and docs change **together**
- one place to find code, docs and history
- the same rules and CI for everyone

 boomchecker-monorepo on GitHub

There is documentation. Read it!



Get Started, top to bottom:

WSL2 ➤ Docker ➤ VS Code

open the devcontainer

share USB with usbipd

your first build

git basics & contributing

Docs are Markdown in docs/. When you add something others will use, document it in the same PR. Preview with `task docs:serve`.

A screenshot of the Boomchecker documentation website. The header is dark blue with the Boomchecker logo and navigation links: Home, Get Started, Firmware, Apps, and Scripts. The main content area has a light blue sidebar on the left with a list of links: Get Started, Prerequisites, Install WSL2, Install Docker Desktop, Install VS Code, Open the devcontainer, Share USB with usbipd, Your first build, Git basics, Monorepo rules, and Contributing (theses). The main content area on the right is titled "Get Started" and contains a welcome message, a section "What you're getting into", and a section "What is a monorepo?".

Boomchecker

Home Get Started Firmware Apps Scripts

[Get Started](#)
[Prerequisites](#)
[Install WSL2](#)
[Install Docker Desktop](#)
[Install VS Code](#)
[Open the devcontainer](#)
[Share USB with usbipd](#)
[Your first build](#)
[Git basics](#)
[Monorepo rules](#)
[Contributing \(theses\)](#)

Get Started

Welcome. This section takes you from a **fresh machine** to a **working build** of the Boomchecker project, and teaches you the rules of the repository along the way. Read it in order – each page assumes you finished the previous one. No prior experience with git, Linux, or embedded development is assumed.

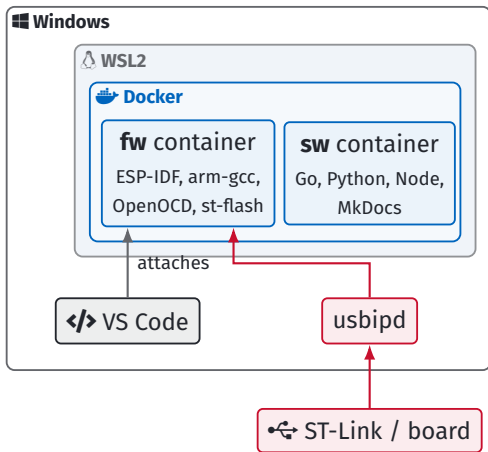
What you're getting into

All Boomchecker work – firmware, backend services, signal-processing experiments, and hardware – lives in **one repository** (a *monorepo*). You will work *inside* that repository: clone it, make changes on your own branch, and open a Pull Request to merge them. That whole flow is what these pages set up.

What is a monorepo?

A **monorepo** ('mono' = one) is a single git repository that holds *all* the projects of a system together, instead of scattering them across many separate repositories (the "polyrepo" approach). In our case, one repository contains the firmware, the backend, the scripts, the

Devcontainers: the same toolchain for everyone



- **No manual toolchain install.**
Reopen in Container and you're done.
- **fw:** firmware for STM32 and ESP32, including flashing and debugging.
- **sw:** apps, scripts and docs.
- **usbipd** forwards the ST-Link / USB serial port from Windows into WSL and then the container.
- “Works on my machine” becomes “works in the container”.

Taskfile: one entry point for commands

Instead of remembering long commands, we write them once in a `Taskfile.yml`:

```
$ cd fw/bom-stm32node
$ task -l          # what can I run here?
$ task build       # build the firmware
$ task flash       # flash over ST-Link
$ task monitor     # talk to the board
```

Docker + Task = tools that save you time

- the environment is defined by the devcontainer
- the commands are defined by Taskfiles
- you focus on your thesis, not on setup

Add a task for anything you run more than twice.

Working on our sensor unit?

Build on the **current firmware** in fw/bom-stm32node and add your part: a driver, a module, a task.

Don't start a fresh CubeMX project. We want one firmware that can do everything.

Something standalone?

projects/<year>-<name>-<topic>/
for your own experiments, reports and datasets.

scripts/: reusable DSP / analysis tools
apps/: backend and web services

Not sure where something belongs? **Ask before you write 2000 lines.**

Branch from an up-to-date main:

```
git switch main && git pull
git switch -c yourname/short-description
```

Commit small, logical steps:

```
git commit -m "feat(stmnode): add IMU driver"
git push -u origin yourname/short-description
```

Conventional Commits

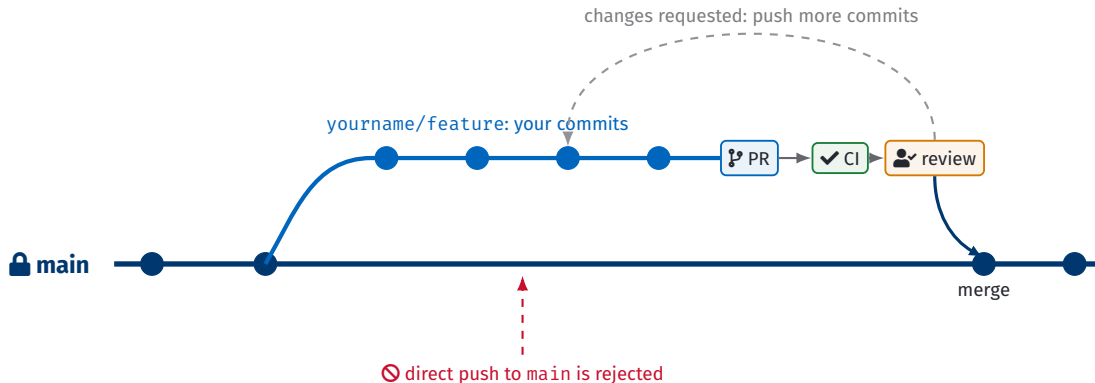
`type(scope): summary`

<code>feat</code>	new feature
<code>fix</code>	bug fix
<code>docs</code>	documentation only
<code>refactor</code>	code change, same behaviour
<code>test</code>	tests
<code>chore</code>	tooling, housekeeping

From our history:

```
feat(stmnode): drone SVM detector
feat(boomlink): fleet discovery
docs: add LoRa section
```

Protected main and pull requests



main is protected: no direct pushes, every change lands through a reviewed PR.

Branch protection



TODO screenshot

branch protection rule for main

`figs/branch-protection.png`

A pull request



TODO screenshot

PR with green checks + approval

`figs/pr-checks.png`

Every PR is built and tested automatically by **GitHub Actions**:

- `test-bom-stm32node-build`: STM32 firmware
- `test-boomdetect-build`: detection library + tests
- `test-boomlink-protocol-build`: LoRa protocol
- `test-api-be-build`: Go backend
- `test-stm32node-cli`: Python host tool
- `docs-pages`: docs build (strict)

Red check = no merge

If CI fails, open the log, fix it, and push again. The PR updates by itself.

Before you ask for review

- it builds in the devcontainer
- the PR says *what, why, how to test*
- no build output, secrets or unrelated files

Metodický pokyn č. 5/2023: Rámcová pravidla používání umělé inteligence na ČVUT...

version V02, in force since 19 Feb 2024 cvut.cz/legislativa-tykajici-se-studia

General principles (Art. 1, 3)

- CTU **supports** responsible, ethical use.
- Stay **independent**: you must reach the same results without AI.
- AI use must be **clearly declared and described**.
- **The author is responsible** for all AI errors (false claims, non-existent literature).
- No personal, sensitive or NDA data: the provider sees everything.

Programming (Art. 4.2): partially

- Follow your supervisor's rules.
- AI saves time and can even propose a whole solution.
- **The student is the author of the code**: know exactly what the generated code does and be able to modify it.
- You must be able to write the same program **without** these tools.

AI in the thesis text (Art. 4.1)

Activity		Note
grammar check	✓ yes	no need to declare
rewording, editing	✓ yes	be critical, it can change the meaning; list the AI tool among the software used
finding authors, institutions	✓ yes	verify, or consult your supervisor
literature search	⓪ partially	inspiration only, never the only source; AI hallucinates, verify everything
text structure	⓪ partially	you are the author and responsible for it; list the AI tool
machine translation	⓪ partially	only from/to a language you speak yourself
theses, results	✗ no	must be your own; text you cannot explain means a disciplinary procedure
citations	✗ no	AI often invents sources; verify every single one

✓ Good use

- explaining a compiler or runtime error
- boilerplate: config, parsers, test scaffolding
- refactoring or optimizing code you wrote
- explaining unfamiliar code line by line (HAL, libraries, our firmware)
- a second opinion on your design

✗ Not OK

- generating the whole assignment and submitting it
- submitting code you cannot explain
- AI-written results or invented citations

At the defense you may be asked to explain **any line**. Not knowing it can mean failing and a disciplinary procedure.

Our stance: use it, but stay the engineer

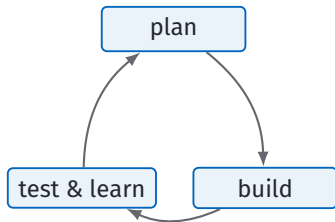
- What you should learn here: **reading and navigating real code**, i.e. firmware, drivers, DSP.
- You **may generate code**, but you must understand it and think it through.
- Review AI output like a PR from a colleague you don't fully trust.
- If a thesis were *about* agentic development, the rules would differ. These theses are not.

Available in the devcontainers

Claude Code and Codex come pre-configured, and `.claude/skills` holds a few shared skills.

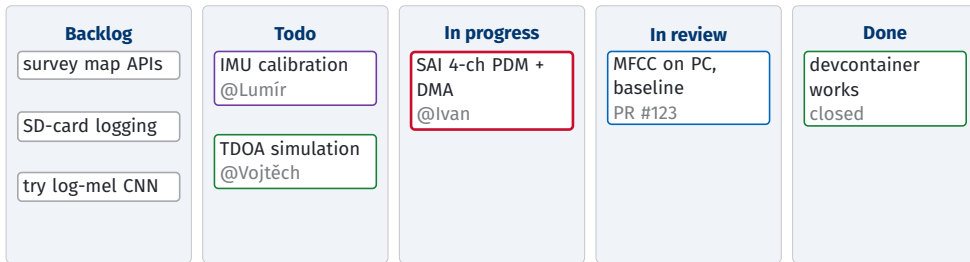
Mention the AI tools you used in the thesis's list of software.

- **Iterate:** small steps, each one working and testable.
- **Make work visible:** everyone sees what's in progress and what's blocked.
- **Limit work in progress:** finish one thing before you start three new ones.
- **Adapt:** a dead end is a result. Record it and change the plan.



No sprints, no ceremonies. Just a **Kanban board** and consultations when you need them.

Kanban on GitHub Projects



- one piece of work = one **issue**, assigned to you
- **you manage your own issues**: create, split, move and close them as you work
- PR description with Closes #N closes the issue on merge

Live demo

create issue, then branch, PR, merge, card moves to Done

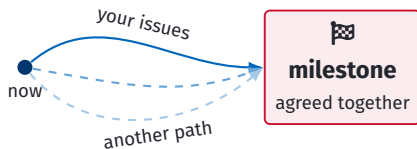
Example issue

Channels from the 4 PDM mics are not aligned

Problem: after de-interleaving, a clap shows up with a different delay in each channel.

Possible approach (optional): check the SAI slot order and the DMA half/full buffer handling, then measure with a common test signal.

An issue only needs to **describe the problem**.
How to solve it is a hint at most.



- **Milestones:** the goals we agree on together.
- **The path is yours:** which issues, in which order.
- **Consult it along the way** so you don't spend weeks in a dead end.

Next steps

1. **GitHub access:** send your GitHub username to Martin (maxamart@fel.cvut.cz).
2. Go through **Get Started** and have the devcontainer running before the first consultation.
3. Make your **first PR**: e.g. fix something unclear in the docs.
4. Agree on your **milestones** with your supervisor, then create your issues.
5. Book a **consultation** with your supervisor.

Dates

📅 mid-semester meeting: **TBD**

📅 final meeting: **TBD**

Contacts

💬 project Discord

✉ Martin Maxa, Kamil Herman, Jakub Svatoš

Questions?

docs.boomchecker.cz